VERSION 2.5.0

HiBench Next

Technical and scientific user manual

Web workflow, configuration and execution

Scala / PySpark comparisons

Evidence, reproducibility and interpretation

Simple English edition 1.0 | 15 October 2026

Mariano Garralda-Barrio

Independent Researcher

© 2026 Mariano Garralda-Barrio

A practical guide for engineers, researchers and students.

Contents

Follow chapters 1-12 for the main web workflow. Use later chapters for configuration, scientific analysis, automation and support.

1. Purpose and scope 3
2. Understand the system 4
3. Install with Docker Compose 5
4. Start from a source checkout 6
5. Find your way around the web 7
6. Manage the Spark laboratory 8
7. Configure the target and storage 9
8. Build your first experiment 10
9. Review execution options before Run 11
10. Use Runs: History and Monitoring 12
11. Compare Scala and PySpark 13
12. Read the comparison correctly 14
13. Edit shared Spark and HiBench settings 15
14. Edit workload parameters 16
15. Choose a data scale 17
16. Catalog: micro, SQL and graph workloads 18
17. Catalog: machine learning and statistics 19
18. Resources, cores and numerical threads 20
19. Interpret timing and Spark metrics 21
20. Study raw task profiles 22
21. Design a reproducible comparison 23
22. Automate with the CLI 24
23. Automate with the REST API 25
24. Export evidence and protect state 26
25. Security and common problems 27
26. Checklists and glossary 28
27. Environment settings reference 29
28. Version evidence and further reading 30
29. Copyright, attribution and document record 31

© 2026 Mariano Garralda-Barrio

1. Purpose and scope

HiBench Next is a benchmark suite for Apache Spark batch workloads. Use it to prepare datasets, run workloads, record execution evidence and compare observations. The web interface, REST API and command line use shared application services.

This manual is for engineers, researchers and students. It uses short English sentences. The web control names stay in English, exactly as they appear in version 2.5.0. A workload is a computational task, such as sorting, WordCount, PCA or regression. An experiment selects its data, implementation, resources and execution conditions.

What version 2.5.0 provides

- 24 workload identifiers with Scala and PySpark implementations.
- Shared Spark-based dataset generators written in Scala.
- A web workflow for configuration, execution, monitoring and comparison.
- REST and CLI access, persistent run records and downloadable evidence.
- Six named data scales and explicit workload parameters.

The validated reference stack is Spark 3.5.9, Scala 2.12.18, Java 11 and Hadoop 3.3.6. The controller needs Python 3.11 or later. A workload Python environment is separate from the controller environment. Spark 4 and Scala 2.13 are outside this release's supported reference.

Read this before the first run

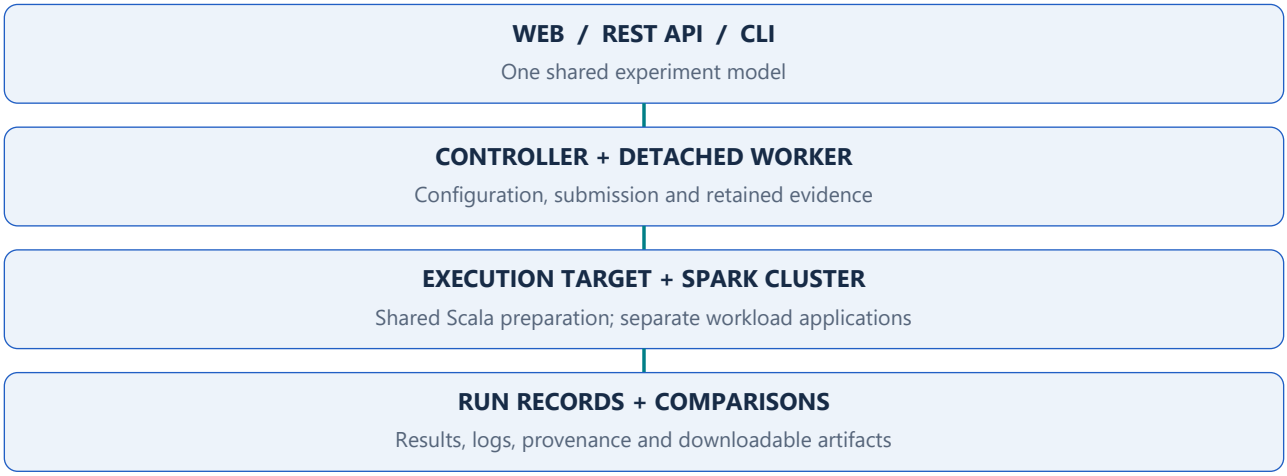
Start with one workload and Tiny. Large is not a promise that your cluster has enough memory. A completed run is not a proof of correct numerical output. The comparator does not certify dataset content, output equivalence or statistical equivalence.

Screenshots in this manual come from an isolated copy of the version 2.5.0 web interface. They show the real WordCount Tiny release smoke test, or an unsubmitted draft. No new benchmark was started to make these screenshots. Screen layout may change with window size. Use the control names as your guide.

© 2026 Mariano Garralda-Barrio

2. Understand the system

The controller and the Spark cluster are different systems. The controller manages experiments. The cluster performs distributed work and stores distributed data. Running the API and web does not automatically create a Spark cluster.



Component	Main role
Web UI	Configure experiments and inspect stored results.
REST API	Integrate the same services with scripts and other tools.
CLI	Use shared services from a terminal.
Detached worker	Run preparation and workload phases outside a web request.
Execution adapter	Reach a Docker container, SSH host or Local Linux host.
Spark cluster	Execute Spark applications using its manager and available resources.
Persistent state	Keep SQLite records, events, logs, provenance and comparisons.

Three different identities

A controller run has a run ID. It can contain preparation and more than one workload application. Each Spark application has its own application ID. A saved comparative analysis has a comparison ID. Do not use these IDs as if they were the same object.

With Scala and PySpark selected, one controller run can contain independent observations for both implementations. Shared data does not mean shared timing or one shared Spark application.

© 2026 Mariano Garralda-Barrio

3. Install with Docker Compose

Use Docker Compose for a self-contained controller build. The host needs Git and Docker with Linux containers and Compose. Java, Maven and Python are provided inside the build images. Dataset and image storage can require substantial disk space.

1. Obtain the companion hadoop-spark-cluster project at its compatible spark-3.5.9 tag. Do not assume its moving main branch is compatible with this release.

2. Start the companion cluster using its own documentation. Verify that Spark, YARN and HDFS are available.

3. Obtain HiBench Next version 2.5.0. Review the target paths in conf/hibench.conf and storage in conf/hadoop.conf.

4. Create deployment/.env from deployment/.env.example. Set a generated API token. Keep this file outside version control.

5. Build and start the controller from deployment/.

```
cd deployment
cp .env.example .env
# Edit .env and set HIBENCH_API_TOKEN.
docker compose --env-file .env -f compose.yml up -d --build --wait
```

In PowerShell, use Copy-Item .env.example .env instead of cp if needed. Generate a token with the documented Docker command:

```
docker run --rm python:3.11-slim python -c "import secrets; print(secrets.token_urlsafe(32))"
```

Open the web at http://127.0.0.1:8501 and API documentation at http://127.0.0.1:8000/docs. Ports can be changed in .env. These URLs are defaults; they are not the temporary ports used to capture this manual.

In Experiments, use Update target, then Check connection. Building the controller image and copying HiBench to the submission target are separate actions.

© 2026 Mariano Garralda-Barrio

4. Start from a source checkout

Use this option for development or a native controller. Keep the source checkout: conf/, launch scripts and workload artifacts are part of the working environment.

```
# Use Java 11 and Maven 3.9.
mvn -B clean package
python -m venv .venv
# Activate the virtual environment for your operating system.
python -m pip install -e .
python -m hibench.launcher --with-ui
```

On Windows, bin/build-java11.ps1 is an alternative build helper that uses Docker. A host with several JDKs must select Java 11 explicitly. Compiled JAR files are generated build artifacts.

Command	Starts
python -m hibench.launcher --with-ui	API and supervised web process.
hibench-serve --with-ui	The same unified launcher.
python -m hibench.launcher	API only.
python -m streamlit run hibench/ui/app.py	Web only, from a checkout.

The web calls shared Python services directly. It does not need to send HTTP requests to its own API. Do not start another service on an occupied port. Use --port and --ui-port when needed.

Configure the Python environment

The controller dependencies are declared in pyproject.toml. There is no separate pysparkbench/requirements.txt to maintain. The Spark submission host and executors must also have a suitable Python environment for PySpark. NumPy, pandas and PyArrow requirements must match the workload deployment. Installing a library only in the controller does not install it on remote workers.

Stop carefully

Stopping the API is not the same as cancelling a detached benchmark. Use Cancel run for an active experiment. Stopping the cluster interrupts Spark work. Keep run records and reconcile interrupted states after recovery.

© 2026 Mariano Garralda-Barrio

5. Find your way around the web

The left sidebar separates daily work, shared parameters and infrastructure. Configure the infrastructure once, then use Experiments and Runs for most tasks.

Area	Page	Use it for
WORKSPACE	Experiments	Select data and implementation, review, then submit.
WORKSPACE	Runs	History, Monitoring and Comparisons.
WORKSPACE	Help	Short workflow and troubleshooting guidance.
PARAMETRIZATION	Spark configuration	Common Spark settings and scale sections.
PARAMETRIZATION	HiBench configuration	Benchmark, generator and monitoring defaults.
PARAMETRIZATION	Workloads	Input shapes, seeds and algorithm parameters.
SPARK INFRASTRUCTURE	Target & storage	Submission connection, runtime paths, storage and profiles.
SPARK INFRASTRUCTURE	Cluster	Existing lab profile and explicit cluster operations.
PROJECT	About	Version, attribution, notices and API links.

Decide where a change belongs

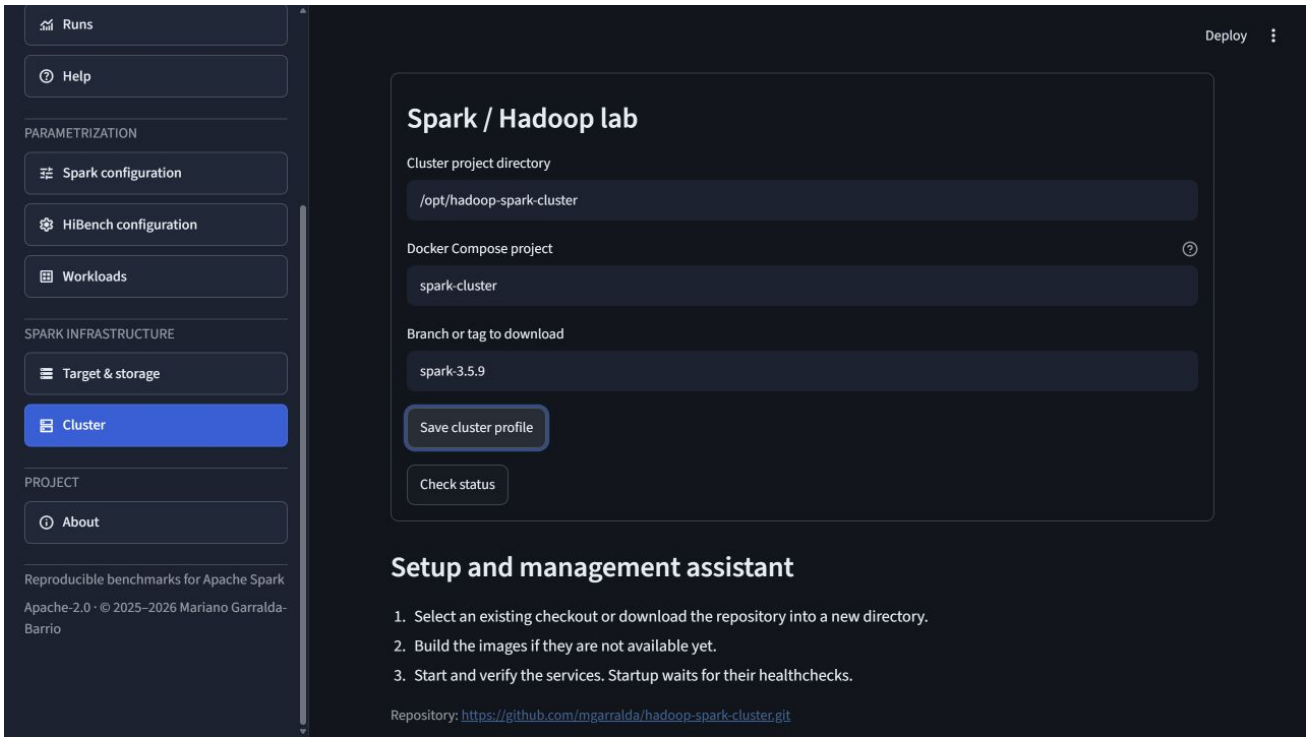
Change a shared default in Parametrization when it should apply to later experiments. Use an Experiments override when it belongs only to the current draft. A saved infrastructure profile describes connection and resource settings; an experiment also contains the workload selection.

Loading a configuration or reusing a run opens an editable draft. It does not submit a benchmark. Monitoring and Comparisons inspect evidence; they do not start a new run.

© 2026 Mariano Garralda-Barrio

6. Manage the Spark laboratory

Open Spark Infrastructure > Cluster to manage a companion Docker laboratory. The page has a project directory, a Compose project name and a branch/tag field. These identify the lab checkout; they do not replace the experiment target settings.



Cluster page with the compatible tag entered in a draft. No download or cluster operation was started.

Operation sequence

1. Select an existing project directory, or download a new checkout.

2. Select a compatible revision. For the validated release, use spark-3.5.9.

3. Build images when required.

4. Start and verify services. Read the latest operation log.

5. Configure Target & storage for the running submission host.

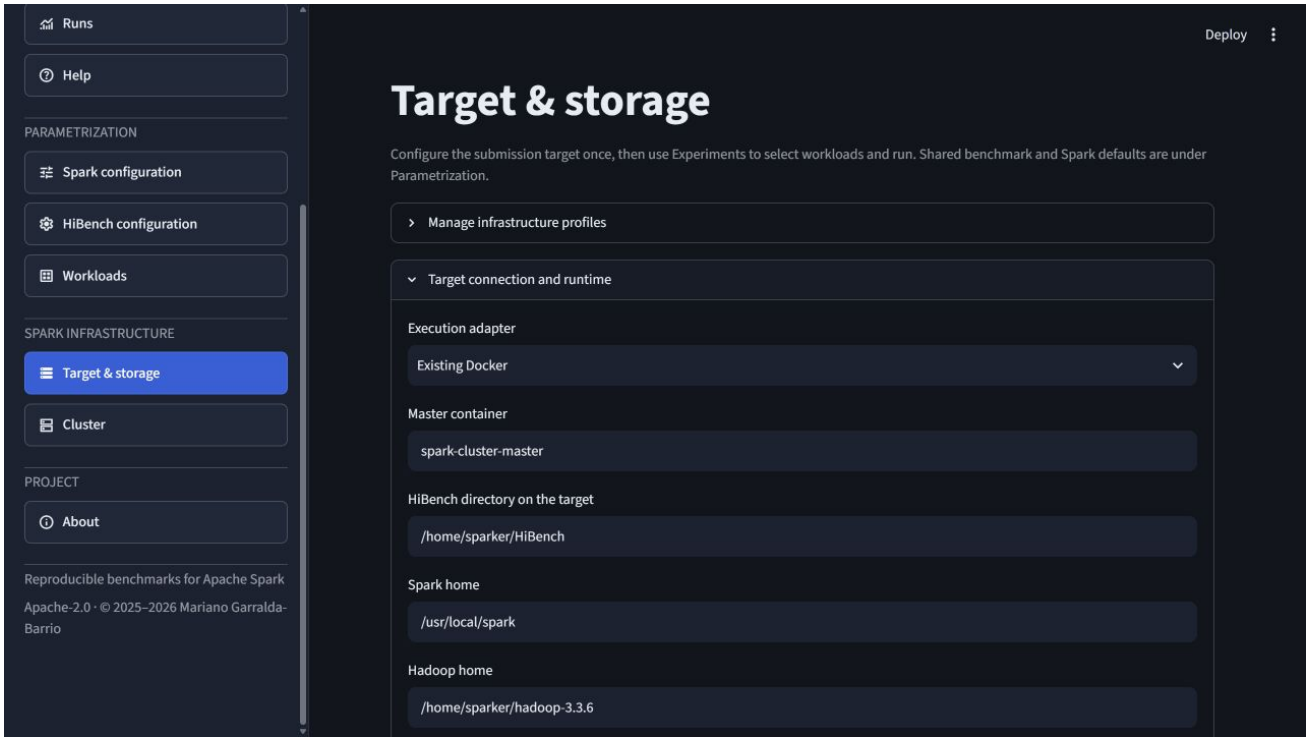
Check status reads the existing Compose project. Stop cluster and Remove containers interrupt cluster services. The normal operations preserve volumes, but they do not preserve an active computation. Do not remove data volumes as a routine cleanup step.

The displayed lab resource estimate is an environment estimate, not the controller's memory need or a universal benchmark requirement. When the controller runs inside Docker, a project directory is a path inside that controller unless it is mounted from the host.

© 2026 Mariano Garralda-Barrio

7. Configure the target and storage

Open Target & storage. An adapter tells HiBench how to reach the submission host. It is separate from Spark's execution mode.



Target connection, runtime paths and distributed storage settings.

Adapter	Meaning
Existing Docker	Use docker exec in an already running Linux container.
SSH	Connect to an existing Linux submission host.
Local Linux	Submit directly on the local Linux host.
Planned adapters	Visible future integrations; not working destinations in 2.5.0.

Enter the target HiBench directory, Spark home, Hadoop home and worker names. These are target paths, not Windows source-checkout paths. Apply target settings before checking the configured endpoint.

Set the storage endpoint, dataset base URI and output base URI. The controller creates isolated subdirectories. Changing the storage location can prevent reuse of datasets prepared at the old location. HDFS access must work from the submission host and the Spark workers.

For SSH, use an authorized key/agent or the controller's HIBENCH_SSH_PASSWORD environment variable. Configure a verified known_hosts file. An explicit identity file takes precedence. Credentials do not belong in experiment JSON or screenshots. Paths from a Docker controller must exist inside that controller.

© 2026 Mariano Garralda-Barrio

8. Build your first experiment

Use one small test to check the complete path before using large datasets. WordCount Tiny is a practical starting point.

1. Open Experiments. Confirm the target and select YARN with Client deployment for a comparable first Scala/PySpark pair.

2. Use Update target after the first build or a runtime change. It copies saved configuration, launch scripts, PySpark modules and the built JAR to the target. It does not start a benchmark.

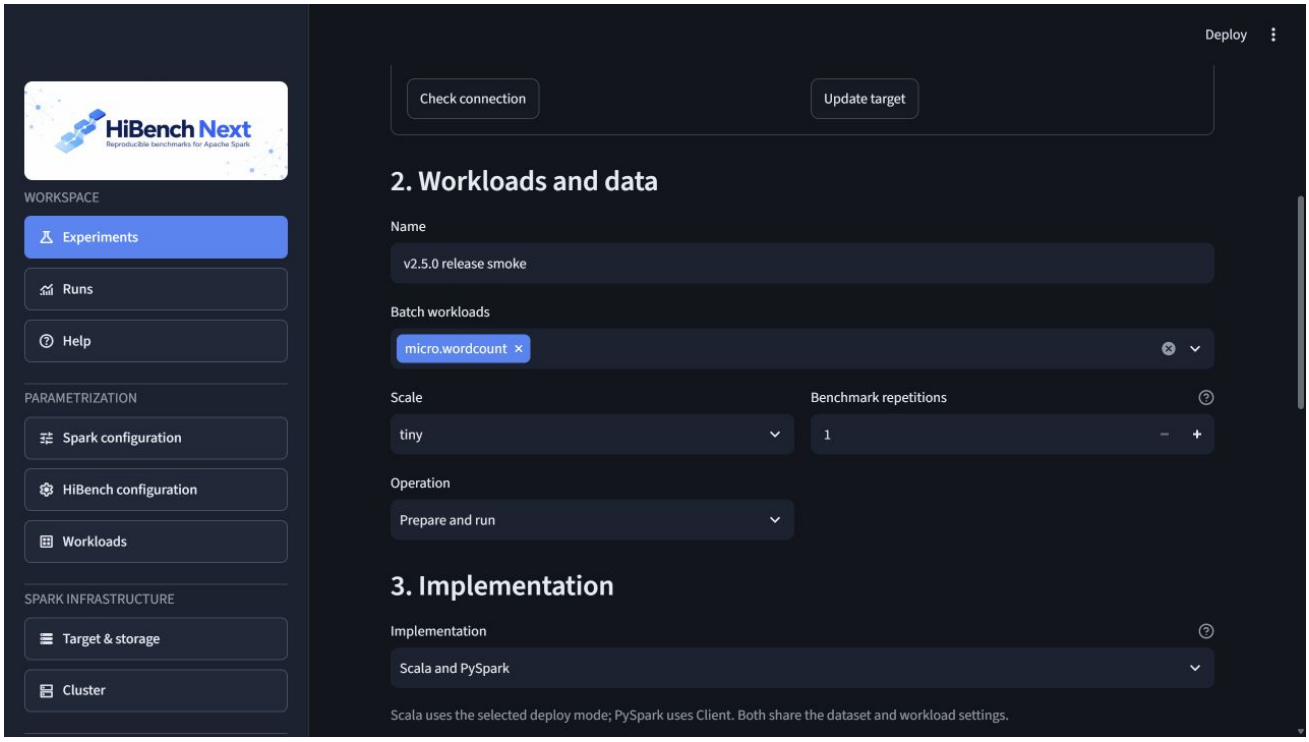
3. Use Check connection. Read any failure before continuing. A past successful check can become invalid when local files or target settings change.

4. Enter a clear name, such as WordCount Tiny baseline.

5. In Batch workloads, keep only micro.wordcount. The initial draft may select many workloads; inspect the list.

6. Select tiny, Prepare and run and one repetition.

7. Select Scala and PySpark. Review the effective settings, then use Run.



An unsubmitted WordCount Tiny draft with both implementations selected.

The example screenshot was opened with Reuse configuration from a completed run. It is a draft, not evidence that the Run button was pressed again. The same input and workload settings are shared by both implementation paths. Preparation remains Scala.

© 2026 Mariano Garralda-Barrio

9. Review execution options before Run

Operation and repetition

Operation	Behavior
Prepare and run	Prepare or reuse compatible data, then execute the selected workloads.
Prepare only	Prepare data without the benchmark execution phases.
Run prepared data	Execute using a compatible prepared dataset; inspect missing-data errors.

Repetitions repeat workload execution after preparation. With both implementations selected, the standard workflow runs Scala then PySpark per repetition. This is not automatic balanced randomization and does not define a warm-up exclusion policy.

Optional controls

Execution options contains a group ID and phase timeout. A group ID is a label for related runs, not proof that their conditions match. Timeout applies to a phase and includes startup. Requested experiment resources contains executor and partition overrides. Resource monitoring configures system sampling. Workload overrides change only the selected experiment. Advanced Spark configuration accepts explicit Spark properties.

The final review lists target, effective driver deployment per implementation, workloads, operation, scale, repetitions and requested resources. Check all of them before Run. Save experiment downloads configuration for later reuse; it is not a dataset export. New run creates a new draft context rather than duplicating measured results.

Manager and deployment

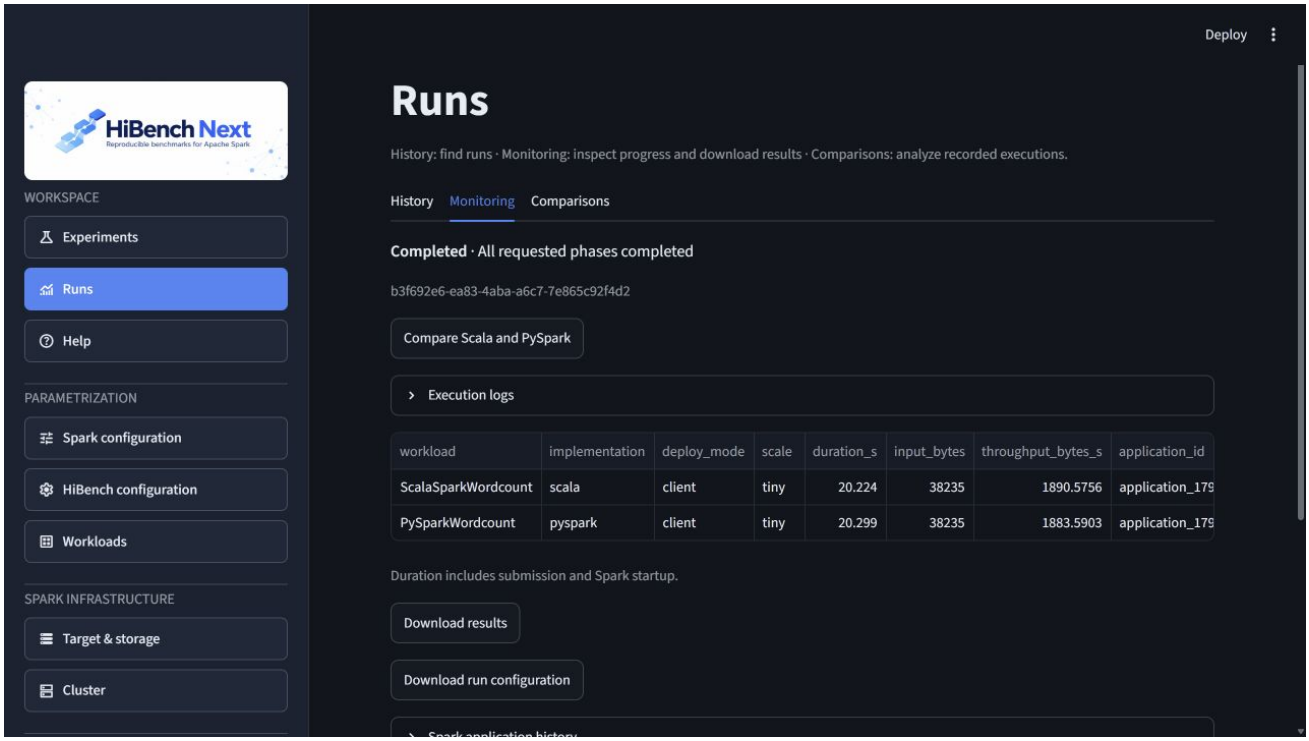
YARN supports Client and Cluster for Scala. PySpark uses Client in HiBench Next. With both selected and YARN Cluster requested, Scala can use Cluster while PySpark uses Client. Standalone uses a spark://host:7077 URL and Client in this release. Local uses local[2] on the submission host and does not use distributed workers.

Inspect spark.master and spark.submit.deployMode in effective evidence. A local-* ID indicates local execution, app-* usually identifies Standalone and application_* identifies YARN. The ID does not identify Client versus Cluster.

© 2026 Mariano Garralda-Barrio

10. Use Runs: History and Monitoring

History helps you find and open recorded runs. A completed run can contain several applications. Keep the controller run ID and each application ID when reporting an experiment.



Monitoring for a completed real WordCount Tiny Scala/PySpark run.

Monitoring shows status, results and expandable evidence. Execution logs contains console output. Spark application history provides History Server links and event-log downloads. Resource monitoring contains node samples and reports. Reproducibility and export exposes provenance, the experiment bundle and reusable configuration.

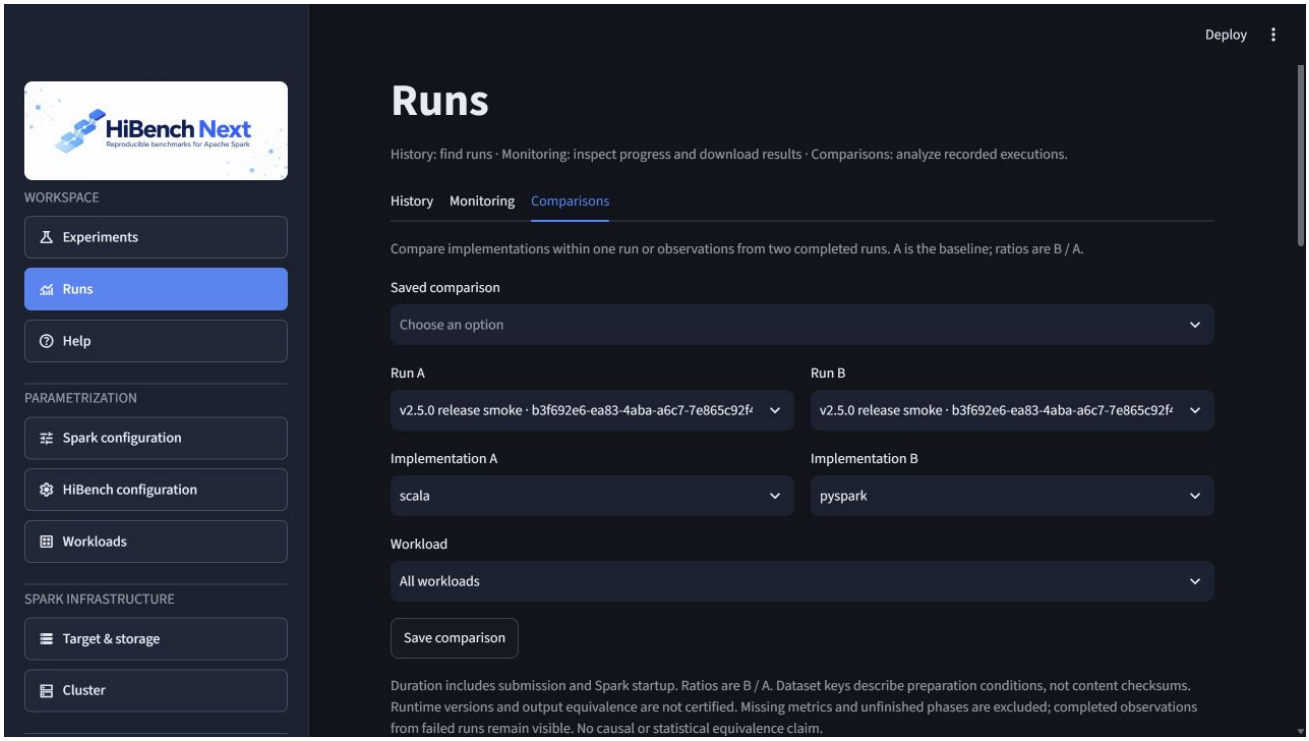
State	Meaning
Queued / Waiting for target	Work is not yet executing its active phase.
Running	A worker is processing the experiment.
Completed	All requested phases completed.
Failed	Inspect the failed phase and logs. Earlier observations can still exist.
Cancelled	Cancellation was processed; data and records can remain.

Cancel run requests cancellation of the active phase and associated Spark application. It can take time. Reuse configuration returns to Experiments without submitting work. Do not treat monitoring as an execution screen.

© 2026 Mariano Garralda-Barrio

11. Compare Scala and PySpark

Open Runs > Comparisons, or use Compare Scala and PySpark on a completed multilanguage run. A is the baseline and B is the comparison side.



A and B selection for the same real run, with a different implementation on each side.

1. Select Run A and its implementation.

2. Select Run B and its implementation. Both sides may use the same controller run.

3. Select one workload, or inspect the full selection.

4. Read Conditions before reading the ratio.

5. Inspect individual observations, sample counts and the duration summary.

6. Save comparison to preserve an analysis snapshot. Download the required format.

Same, Different and Unknown describe recorded condition values. Unknown is not Same. Dataset keys describe preparation conditions rather than verified content. Runtime versions and output equivalence are not certified by this comparison service.

Implementations are not pooled into one median. Unfinished phases and invalid positive durations are excluded. Completed observations from failed or cancelled parent runs can remain visible, with a warning. Inspect exclusions instead of silently dropping failures.

© 2026 Mariano Garralda-Barrio

12. Read the comparison correctly

The main duration includes submission and Spark startup. A ratio is B median divided by A median. Relative difference is 100 times the ratio minus one. A ratio above one means B took more time; it does not identify why.

B / A < 1	B / A = 1	B / A > 1
B took less time	Same summary duration	B took more time

Real release-smoke example

Observation	Scala A	PySpark B
Workload / scale	WordCount / Tiny	WordCount / Tiny
Duration including startup	20.224 s	20.299 s
Included observations	1	1
Effective deployment	Client	Client

For this pair, B/A is about 1.0037 and the relative difference is about +0.37%. The standard deviation is unavailable with one observation; it is not zero. This example illustrates how to read the screen. It does not show a stable PySpark overhead or statistical equivalence.

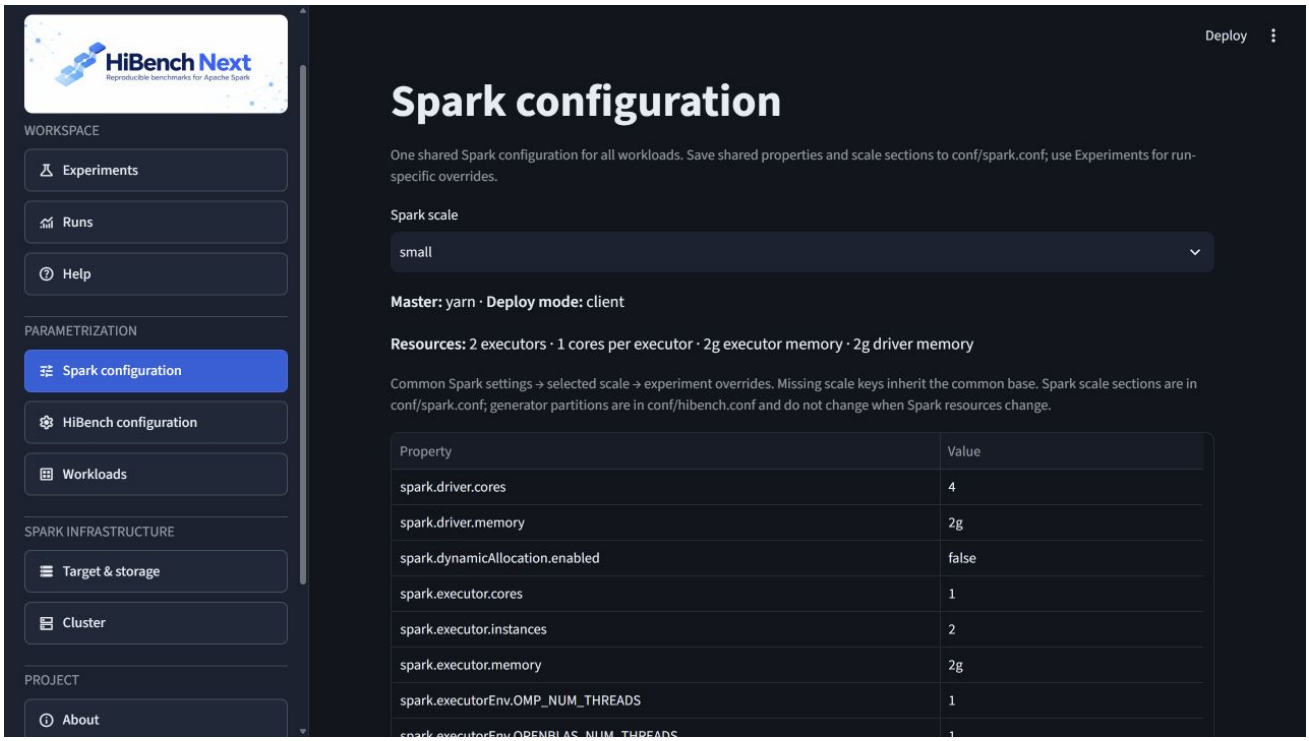
Individual-duration charts show samples and medians. Workload ratio charts use a 1x reference. SVD may also provide protocol-specific phase tables. Compare phases only when their measurement protocol and boundaries match.

Save comparison creates a versioned JSON snapshot with selections, observations, summaries, conditions and source hashes. Reopening can warn that original source records changed or are unavailable. The snapshot preserves analysis evidence; it does not freeze the physical cluster or input files.

© 2026 Mariano Garralda-Barrio

13. Edit shared Spark and HiBench settings

Parametrization owns maintained defaults. Spark configuration edits conf/spark.conf. Common properties are followed by explicit scale sections. Selected-scale properties override common values; experiment overrides have higher priority.



Shared Spark properties and the selected Small resource baseline.

HiBench configuration controls common benchmark settings, generator partitions and monitoring defaults. Generator partitions are separate from Spark SQL shuffle partitions. A change in generator partitions can change random streams and dataset layout; it is more than a performance setting.

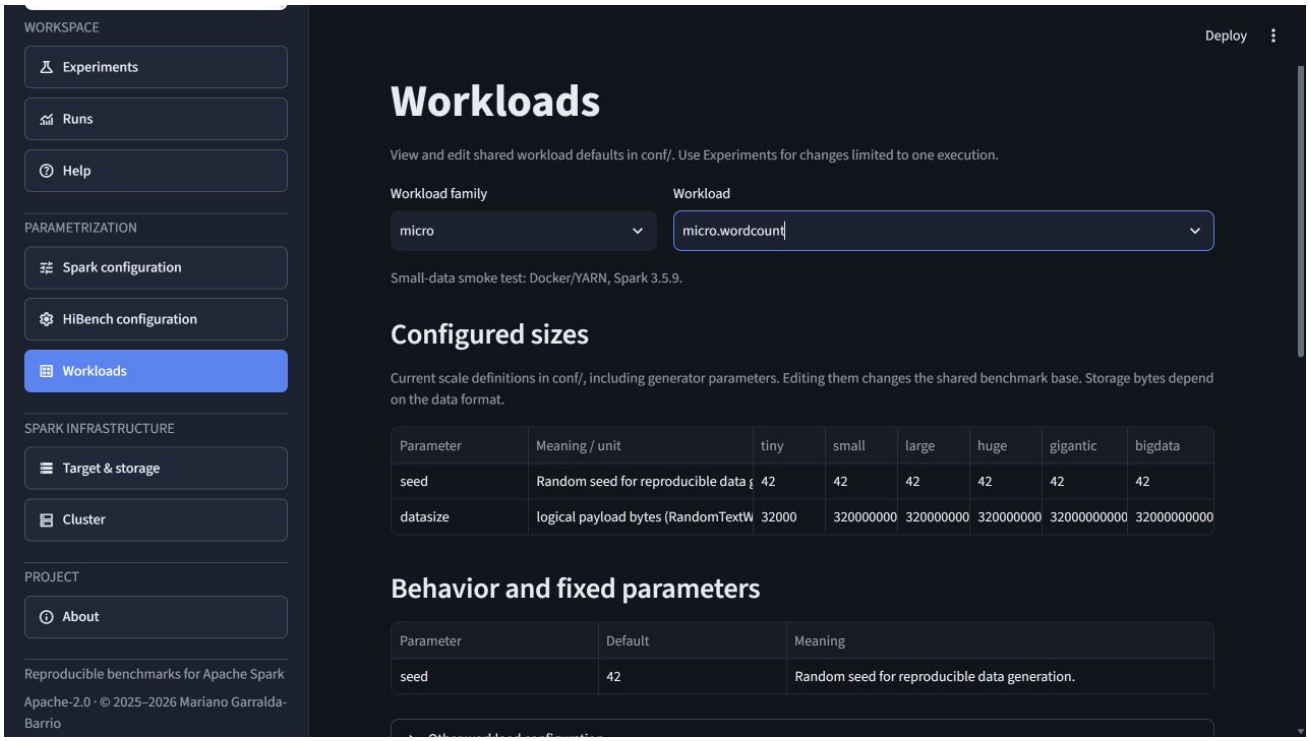
Save and update target saves canonical files and synchronizes the target selected in Experiments. If target installation fails, the local save remains and can be retried. Shared changes and target updates are blocked while controller runs are queued or active.

An existing exported experiment can contain explicit overrides. Changing a shared default does not silently rewrite those historical conditions. Reopen and inspect the resolved draft if you want to use the new defaults.

© 2026 Mariano Garralda-Barrio

14. Edit workload parameters

Open Workloads under Parametrization. Select a family and workload. Configured sizes shows the six scale definitions, units and generator parameters. Behavior and fixed parameters explains settings that do not belong to one scale.



WordCount scale definitions and the shared seed. Some columns need horizontal scrolling.

Edit shared defaults changes the authoritative workload file. Scale-specific edits affect the chosen scale; fixed parameters can affect every scale. Save and update target also updates the selected submission target.

For one experiment only, use Workload overrides for this experiment in Experiments. Keep the algorithm, seed, dimensions, distribution and materialized output contract explicit. The same workload parameters apply to Scala and PySpark unless a documented implementation-specific parameter is needed.

Do not compare by a label such as Large alone. Record the actual row count, dimensions, component count, iteration limit and output behavior. Historical saved runs can use old definitions of that label.

© 2026 Mariano Garralda-Barrio

15. Choose a data scale

Tiny is a smoke-test tier. Small through Bigdata usually increases the main input count by ten. Workload families have different volume units and computational costs. Equal scale names do not mean equal runtime, storage or memory.

Family / input	Tiny	Small	Large
Sort / WordCount logical text	32 KB	320 MB	3.2 GB
TeraSort / HDFS Repartition records, 100 B	320	3.2 million	32 million
Dense native ML rows x features	100 x 100	400,000 x 1000	4 million x 1000
SVD rows x features	100 x 100	50,000 x 1000	500,000 x 1000
KMeans / GMM samples x features	1000 x 3	50,000 x 20	500,000 x 20
SQL requested pages / visits	120 / 1000	12,000 / 100,000	120,000 / 1 million
PageRank pages	50	5000	50,000
Bayes documents	25,000	30,000	300,000
LDA documents	100	1000	10,000
ALS ratings	2000	20,000	200,000
NWeight requested pairs	10,000	100,000	1 million
Sleep seconds per task	1	3	10

Dense native ML covers Linear, LR, SVM, RF, GBT, PCA, Correlation and Summarizer. Decimal byte estimates count feature doubles only: 3.2 GB at Small and 32 GB at Large. SVD Large is about 4 GB of feature doubles. IDs, labels, encoding, models and workspace add other costs.

Huge, Gigantic and Bigdata continue the count ladder, but they do not have universal capacity or duration validation. Sleep uses 30, 60 and 90 seconds per task instead. Native ML dimensions stay fixed from Small upward. SQL visits are requested counts; filtering can reduce actual rows. In-memory Repartition uses records per generation partition, not a global record count.

Consult docs/workload-scales.md and the active conf/ files for the full six-tier definitions. Linear/PCA Small/Large and SVD Large have selected timing pilots. This does not certify every workload or every larger tier.

© 2026 Mariano Garralda-Barrio

16. Catalog: micro, SQL and graph workloads

All entries have Scala and PySpark execution paths. Data preparation is shared. The description below gives task purpose, not a promise of identical historical implementation behavior.

Identifier	Purpose and main observation
micro.sort	Sort text data; inspect scan, ordering, shuffle and output behavior.
micro.terasort	Sort fixed-record data under its binary/Parquet contract.
micro.wordcount	Tokenize and count words using structured Spark operations.
micro.repartition	Redistribute records. HDFS and in-memory generation modes have different input contracts. Cache and output settings change what is measured.
micro.sleep	Request sleep in tasks; a synthetic scheduler and process-boundary control.
sql.scan	Scan typed query input.
sql.aggregation	Group and aggregate the SQL dataset.
sql.join	Join the related SQL tables.
websearch.pagerank	Iterative page-rank propagation through structured operations. Record the iteration count.
graph.nweight	Weighted graph propagation with explicit top-K behavior. It is not a claim to reproduce defects in the legacy implementation.

Materialization matters

Reading an input, caching it, counting it, shuffling it and writing a result are different actions. A disabled output sink changes the workload boundary. A lazy transformation does not prove that all transformed rows were computed. Inspect both source contracts and recorded actions when comparing results.

Repartition volume example

If in-memory datasize is 3.2 million records per partition and generation uses 14 partitions, the total is 44.8 million records. At 200 bytes each, the logical payload is 8.96 GB. Do not interpret the per-partition setting as a global size or confuse cache-accounting bytes with HDFS input bytes.

© 2026 Mariano Garralda-Barrio

17. Catalog: machine learning and statistics

Identifier	Task and important controls
ml.linear	Linear regression; feature width, split, solver/regularization and iteration limit.
ml.lr	Logistic regression; labels, split, optimizer settings and convergence.
ml.svm	Linear support vector classification; labels, split and optimization controls.
ml.rf	Random Forest classification; trees, depth, features and split.
ml.gbt	Gradient-boosted trees; iteration/depth and data contract.
ml.bayes	Naive Bayes; document or configured input mode and vocabulary/labels.
ml.kmeans	KMeans clustering; generated distribution, k and iteration limit.
ml.gmm	Gaussian mixture fitting; feature shape, k and iterations.
ml.als	Recommendation through alternating least squares; users, items, ratings and rank.
ml.lda	Topic modeling; documents, vocabulary, topics and iterations.
ml.pca	Principal components; input shape and requested components. Fitting and lazy projection are different boundaries.
ml.svd	Uncentered singular-value computation through Gram construction and local eigendecomposition; dimensions and requested rank matter.
ml.correlation	Correlation statistics over feature vectors.
ml.summarizer	Feature summary statistics.

Numerical checks need a defined scope

Compare a meaningful observable for the algorithm: coefficients and RMSE, predictions, explained variance, singular values or another stated result. Floating-point reductions can differ slightly. Use justified tolerances and document them.

PCA component signs and singular-vector signs can differ without changing the represented subspace. Checking explained variance alone does not check all projections. Checking singular values alone does not check a full reconstruction. Do not label these limited checks as complete output equivalence.

© 2026 Mariano Garralda-Barrio

18. Resources, cores and numerical threads

Requested resources are not the same as physical capacity. The scheduler must fit executors and memory into available nodes or containers. Two executors with two cores each provide a nominal four concurrent task slots when spark.task.cpus is one, subject to actual scheduling.

Reference scale	Executor request	Driver heap	Declared native threads: driver / executor
Tiny	3 x 1 core, 1 GiB each	1 GiB	1 / 1
Small	2 x 1 core, 2 GiB each	2 GiB	1 / 1
Large	2 x 2 cores, 2 GiB each	2 GiB	4 / 1

These values are shared defaults in conf/spark.conf, not capacity guarantees. Huge and larger sizes need a separate sizing review; larger data names do not automatically allocate a larger cluster. Client-mode driver CPU capacity depends on the submission host/container. spark.driver.cores alone does not enforce that capacity.

BLAS and LAPACK are numerical library interfaces for vector, matrix and decomposition operations. Implementations such as OpenBLAS can use native code and several threads. Breeze/netlib on the JVM and NumPy can use different providers and routines.

OPENBLAS_NUM_THREADS and OMP_NUM_THREADS declare thread limits. They do not measure active threads. Executor variables use spark.executorEnv. Driver declarations use the appropriate driver environment; HiBench also applies its numerical YARN app-master environment settings to Client submission processes.

Avoid oversubscription: several Spark tasks can each start several native threads and compete for the same CPU quota. Increasing executor cores, driver quota or native threads changes different constraints. Change one treatment at a time and record effective values, provider versions and container quotas.

© 2026 Mariano Garralda-Barrio

19. Interpret timing and Spark metrics

Boundary or counter	What it describes
HiBench duration	Submission wall time, including Spark startup and shutdown around the workload phase.
Spark application duration	ApplicationStart to ApplicationEnd in Spark evidence.
Job-interval union	Wall-clock time covered by one or more job intervals; overlapping intervals count once.
SQL execution duration	Registered SQL executions; it may omit other ML jobs.
Sum of task runtime	Overlapping task counters; it can exceed application wall time.
JVM task CPU	CPU accounting of the measured JVM task thread, not total application CPU.
System resource samples	Host/container observations; not isolated task-level Spark counters.

Do not add overlapping counters as if they were separate phases. Total minus SQL is an arithmetic residual, not a measured driver-only phase. Jobs, stages and tasks describe coarse structure; matching counts do not prove matching physical plans.

SVD instrumentation

SVD records explicit timers under svd-phases-v1 and numerical-runtime metadata when available. Inspect startup, distributed Gram work, local eigendecomposition and lifecycle boundaries separately. Parent totals contain other phases. Missing instrumentation is not a measured zero.

Scala and PySpark SVD use different aggregation and local numerical paths. A result can reflect several mechanisms at once. A large SQL or executor-runtime difference alone does not prove Python interpreter overhead. Driver-local numerical work is invisible to a task-only profile.

© 2026 Mariano Garralda-Barrio

20. Study raw task profiles

Use Spark application history to inspect jobs, stages and tasks. The event-log archive is separate from controller console logs and system monitoring. Download Spark event logs while the History Server remains available. Configure spark.eventLog.enabled and a durable event-log directory before the run.

The prior characterization method uses 24 task counters, with a derived tasks-per-stage feature. These cover runtime/CPU, deserialization, results, GC, spill, peak execution memory, input/output, and shuffle volume/locality/wait/write counters. Native units differ: CPU and shuffle-write time use nanoseconds, while many other times use milliseconds. Retain units when exporting.

Start with raw values

1. Identify the application, successful/failed attempts, stage and physical operation.

2. Select a stated ordering, such as Task ID or launch time. Keep attempt identity.

3. Plot raw counter values with units. Show stage boundaries where useful.

4. Report sums, medians or maxima according to the counter meaning.

5. Link an observed difference to the plan before proposing a mechanism.

A task-index profile is not a uniformly sampled time series. Equal indices do not guarantee equivalent tasks. Placement can change remote versus local shuffle even when total shuffle is the same.

Independent min-max scaling removes magnitude. Standardization and cumulative-sum/quadratic descriptors can reveal shape, but can also amplify tiny counters or smooth peaks. Different raw runtimes do not automatically imply different normalized clusters. Define constant-series handling and ordering. This release's web comparator does not automatically produce a full physical-plan or 24-counter comparative analysis; those investigations use exported evidence and separate analysis tools.

© 2026 Mariano Garralda-Barrio

21. Design a reproducible comparison

Define the question before collecting results. A useful question is: which execution paths and task costs differ for the same materialized input under stated resources? It is stronger than asking which language is always faster.

Before execution

- Freeze benchmark source, cluster revision, images and library providers where possible.
- Record input URI, schema, counts, dimensions, seeds, generator partitions and materialization contract.
- Fix manager, deployment, driver/executor resources, native threads, AQE and partition settings.
- Define warm-up handling, repetitions, ordering and failure/exclusion rules.

During and after execution

Use independent application repetitions. Balance or randomize Scala-first and PySpark-first order when studying language variants; the standard both selection is fixed-order. Keep cache conditions consistent. Preserve failures and explain retries. Tasks inside one application are not independent run replications.

Report individual observations and suitable summaries. Five or ten measured pairs can be a practical starting design when feasible, but sample size must match the required precision. One pair supports an exploratory observation only. Do not invent a confidence interval or treat missing deviation as zero.

Distinguish input identity, checked output observables, physical paths and performance. Shared Spark ML can explain similar behavior. SVD, in-memory Repartition and Sleep illustrate different kinds of implementation boundaries. They do not establish a universal ranking or prove that adding PySpark creates 24 new behavior classes.

© 2026 Mariano Garralda-Barrio

22. Automate with the CLI

Run these commands from an installed source checkout with the correct controller environment. Obtain defaults first rather than maintaining a second set of resource values.

```
hibench list
hibench adapters
hibench defaults --scale tiny > experiment.json
# Edit experiment.json: keep only micro.wordcount and set both.
hibench validate experiment.json
hibench resolve experiment.json
hibench install experiment.json
hibench doctor experiment.json
hibench run experiment.json --wait
```

Inspect an existing run

```
# Replace RUN_ID with the returned controller run ID.
hibench status RUN_ID
hibench logs RUN_ID
hibench export RUN_ID evidence.zip
hibench cancel RUN_ID
hibench reconcile RUN_ID
```

Cancel is an explicit action, not a required step after success. Reconcile checks interrupted recorded states; it does not automatically restart the workload. Use --request-key on run when you need submission deduplication under that service's rules.

hibench profiles lists infrastructure profiles. hibench resolve can apply a selected profile through --profile. The CLI compare command exposes its stored-run comparison operation; use the web or /comparisons API for the saved implementation-selected comparative analysis described in this manual. They are not interchangeable output contracts.

The Compose .env file supplies container settings. A native CLI process does not automatically read it; export required environment variables before launch.

© 2026 Mariano Garralda-Barrio

23. Automate with the REST API

Use the API host and port configured for your deployment. Swagger UI at /docs, ReDoc at /redoc and /openapi.json expose the current contracts. Use Authorize when a bearer token is configured. Never place a real token in a published experiment or notebook.

Operation	Endpoint below /api/v1
Read defaults	GET /experiments/defaults?scale=tiny
Read catalog	GET /workloads
Resolve imported overrides	POST /experiments/resolve
Validate a full experiment	POST /experiments/validate
Update target / diagnose	POST /diagnostics/install and /diagnostics/doctor
Submit / list runs	POST /runs and GET /runs
Status / results / logs	GET /runs/{id}, GET /runs/{id}/results, GET /runs/{id}/logs
Cancel / recover state	POST /runs/{id}/cancel or POST /runs/{id}/reconcile
Export a run	GET /runs/{id}/export
Analyze / save comparison	POST /comparisons/analyze or /comparisons
Read / export comparison	GET /comparisons/{id} or /comparisons/{id}/export

The prefix is omitted in the table for readability. Configuration PUT operations save locally. Use the install diagnostic separately to update the selected target. Run-history pagination uses limit and offset.

Minimal Python submission

```
import json, os, urllib.request
base = "http://127.0.0.1:8000/api/v1"
headers = {"Content-Type": "application/json",
           "Authorization": "Bearer " + os.environ["HIBENCH_API_TOKEN"]}
def call(path, body=None):
    data = None if body is None else json.dumps(body).encode()
    req = urllib.request.Request(base + path, data=data, headers=headers)
    with urllib.request.urlopen(req, timeout=60) as response:
        return json.load(response)
spec = call("/experiments/defaults?scale=tiny")
spec["workloads"] = [{"id": "micro.wordcount", "parameters": {}}]
spec["implementation"] = "both"
spec["target"]["deploy_mode"] = "client"
result = call("/runs", {"experiment": spec})
print(result["run_id"])
```

Install and diagnose the target before this submission example. To resolve a compact imported file, send {"experiment": overrides} to /experiments/resolve. Full submission models require explicit resources and monitoring; defaults provides them. HTTP 401 means missing/invalid authentication; 422 commonly indicates validation failure; 503/504 can indicate target or timeout failures. Read the response detail.

© 2026 Mariano Garralda-Barrio

24. Export evidence and protect state

Artifact	Use
Results JSON	Recorded workload observations and available phase metadata.
Run configuration	Full specification recorded for the run.
Editable preset / experiment YAML	Portable settings to import as a new draft.
Experiment bundle ZIP	Configuration, provenance, console logs, metrics and available resource reports.
Spark event-log ZIP	Spark jobs/stages/tasks; separate History Server download.
Comparison JSON / CSV / HTML / ZIP	Saved analysis, raw observations, summary and offline charts.

The default local state is report/control, controlled by HIBENCH_STATE_DIR. It includes runs.sqlite, run directories, dataset manifests and comparisons/<comparison_id>/comparison.json. Your configured state path can differ.

Docker Compose binds host conf/ and report/ into the controller. Set HIBENCH_CONF_HOST_DIR and HIBENCH_REPORT_HOST_DIR in .env to choose host directories. The state directory must stay under the persistent report mount. HDFS input/output and Spark event logs belong to the companion cluster's own storage and volumes.

Backup procedure

1. Wait for active work to finish, or cancel it explicitly and verify the result.

2. Stop the controller service before copying its configuration and report/state directories.

3. Preserve SQLite and any WAL files as a consistent set.

4. Back up cluster datasets and event-log storage separately.

5. Restore compatible paths and versions, then check target and reconcile affected runs.

Container recreation can preserve files but cannot preserve an interrupted process. A report backup alone does not contain the complete distributed dataset.

© 2026 Mariano Garralda-Barrio

25. Security and common problems

The default Compose deployment binds web/API ports to loopback. It is a trusted local administration tool. The API uses a bearer token when configured. Streamlit does not provide user authentication here. A Docker socket allows control of the Docker host; do not expose this setup publicly without a separately secured deployment.

Symptom	First checks
Web will not start	Port conflict, launcher output, Streamlit log, Docker health.
API returns 401	Token, Authorization header, controller environment and restart after changes.
Check connection fails	Adapter access, running container/SSH host, target paths and compatible versions.
Old code runs on target	Rebuild JVM artifacts, then Update target; inspect installed hashes.
Local-* application when YARN was expected	Review Spark execution mode and recorded spark.master.
Out of memory	Confirm actual rows/dimensions, driver/executor limits, native workspace and batching. Start smaller.
No Spark History links/data	Event logging enabled, durable directory, History Server availability and browser-accessible URL.
Dataset not reusable	Storage endpoint/base path or generator settings changed; inspect manifest and input availability.
Similar Scala/PySpark time	Shared JVM core or startup dominance may explain it; inspect phases/plans before a language claim.
Interrupted run remains active	Check the real application and use reconcile; do not submit duplicate work blindly.

Do not fix a host-key error by disabling verification. Use a verified known_hosts entry. Do not fix an experiment by silently changing its data volume while retaining the original scale claim. Keep failed evidence for diagnosis.

© 2026 Mariano Garralda-Barrio

26. Checklists and glossary

Before a benchmark

- Compatible cluster, healthy storage and enough free disk.
- Correct adapter, target runtime and effective manager/deployment.
- One deliberate workload selection; Tiny for the first smoke test.
- Actual shape, generator settings, materialization and resource request reviewed.
- Built/updated target, diagnostics passed and evidence collection enabled.

Before a scientific result is shared

- Run ID, application IDs, source and environment recorded.
- Input properties and output-check scope stated.
- Timing boundary, units, retries and exclusions stated.
- Independent repetitions and uncertainty matched to the claim.
- Data, event logs, configuration and analysis backed up where possible.

Term	Meaning in this manual
Adapter	How the controller reaches the submission host.
Driver	Coordinates a Spark application and may do local numerical work.
Executor	Process that runs Spark tasks.
Job / stage / task	Spark execution units at different levels.
Scale	Named input profile, not a capacity guarantee.
Manifest	Dataset preparation record, not a content checksum.
Provenance	Recorded origin and conditions of an observation.
Raw profile	Ordered, unnormalized counter values with units.
BLAS / LAPACK	Interfaces for numerical linear algebra routines.
Comparison snapshot	Saved analysis derived from independent observations.

© 2026 Mariano Garralda-Barrio

27. Environment settings reference

The Compose template is deployment/.env.example. Copy it to the ignored .env file and keep its secrets local. Relative host paths in this stack are resolved from the Compose directory. Native launches need exported environment variables; they do not automatically load this file.

Setting	Use and scope
HIBENCH_API_TOKEN	Bearer token for API access. Generate a value; do not publish it.
HIBENCH_API_PORT / HIBENCH_UI_PORT	Published host ports; defaults are 8000 and 8501. Internal service ports stay fixed in Compose.
HIBENCH_CONF_HOST_DIR	Host configuration directory; it must contain the existing conf/ files.
HIBENCH_REPORT_HOST_DIR	Host directory for persistent controller state, logs and reports.
HIBENCH_DOCKER_SOCKET	Docker socket mounted into the controller. Treat it as host administration access.
HIBENCH_STATE_DIR	Controller-side state directory. In Compose keep it under /opt/hibench/report.
HIBENCH_UI_HOST	Web listening address inside the controller; the Compose default is 0.0.0.0 with host publication on loopback.
HIBENCH_REPOSITORY	Controller checkout root. The standard image uses /opt/hibench.
HIBENCH_START_UI	Launcher-managed UI setting. Use --with-ui for unified native startup; retain the documented image defaults.
HIBENCH_SSH_HOST / USERNAME / PORT	SSH connection defaults. Use the full variable names HIBENCH_SSH_USERNAME and HIBENCH_SSH_PORT.
HIBENCH_SSH_IDENTITY_FILE	Existing private-key path inside the controller. Mount it read-only where needed.
HIBENCH_SSH_KNOWN_HOSTS_FILE	Independently verified host-key file inside the controller.
HIBENCH_SSH_PASSWORD	Optional password from the controller environment. Explicit identity-file authentication takes precedence.

Do not replace the image's internal paths or ports with host paths. A host bind mount maps the two sides. Keep credentials and local Compose overrides outside the release package. Restart the controller after changing credential environment variables.

© 2026 Mariano Garralda-Barrio

28. Version evidence and further reading

The local v2.5.0 closure passed 195 Python tests, validation of 144 workload/scale configurations, a clean Java 11 Maven build, isolated source/wheel builds and a Docker image build. The built image passed API/web health checks and a real WordCount Tiny Scala/PySpark run with comparison saving, evidence ZIP downloads and configuration reuse.

These are functional/build checks. They do not certify output equivalence for every algorithm, statistical language rankings or capacity at every scale. Selected numerical and scale pilots have their own stated limits. The screenshot data in this manual are from run b3f692e6-ea83-4aba-a6c7-7e865c92f4d2, with application IDs ending 0051 and 0052 under epoch 1791980278777.

Maintained project references

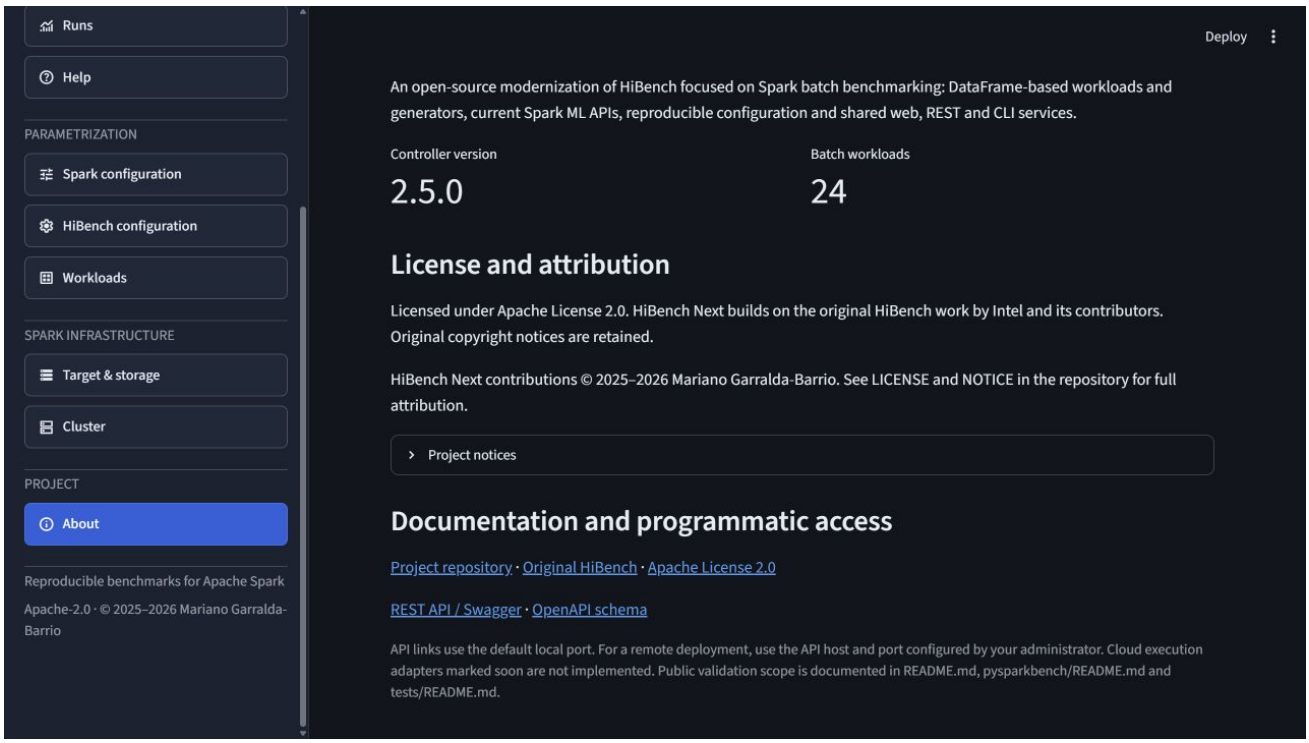
- Project: https://github.com/mgarralda/HiBench-Next
- Root README.md: scope, setup and interface overview.
- docs/configuration-model.md: source of defaults and override rules.
- docs/workload-scales.md: active scale contracts and exceptions.
- deployment/README.md: Compose, persistence, access and SSH.
- sparkbench/README.md and pysparkbench/README.md: implementation and numerical paths.
- tests/README.md and tests/tools/README.md: validation and scientific evidence tools.
- CHANGELOG.md, CITATION.cff, LICENSE and NOTICE: release scope and attribution.

Scientific and upstream context

The original HiBench lineage is maintained at https://github.com/Intel-bigdata/HiBench (archived). This community fork is not an official Intel release. Spark reference documentation: https://spark.apache.org/docs/3.5.9/ . Task-profile methodology: Garralda-Barrio, Eiras-Franco and Bolon-Canedo, Journal of Parallel and Distributed Computing 189 (2024), 104881, https://doi.org/10.1016/j.jpdc.2024.104881 . The manual explains how to use evidence; it does not claim to reproduce all analyses from that paper.

© 2026 Mariano Garralda-Barrio

29. Copyright, attribution and document record



The About page identifies the controller version and project attribution.

Copyright © 2026 Mariano Garralda-Barrio.

HiBench Next 2.5.0 - Technical and scientific user manual. English edition 1.0. Prepared on 15 October 2026 for the version 2.5.0 functional scope.

Original manual text and project artwork are provided under Apache License 2.0, consistent with the project notices. Preserve the copyright, LICENSE and NOTICE when redistributing the relevant materials. Original HiBench and third-party copyrights remain with their respective owners. Apache Spark and other names identify the technologies discussed; no endorsement is implied.

This document describes observed software behavior and reference settings. It provides no warranty of numerical equivalence, runtime, capacity or suitability for a particular environment. Recheck the effective configuration before running costly experiments.

Screenshots were captured from an isolated local interface with copied release-smoke evidence. This edition contains no production credentials. Authoring and evidence organization used AI assistance; the named author remains responsible for review and publication.

Keep this manual with its editable Markdown, build script and screenshot assets. The delivery folder can be moved into the release documentation without changing the application. Regenerate screenshots when controls or navigation change in a later version.